



(/rol/app/)

Menu

Korean

홈 (/rol/app/) (home) > Observing OpenShift Service Mesh

> Chapter 1: Observing OpenShift Service Mesh

Observing OpenShift Service Mesh

Lesson

Lab Environment

Tracing Services With Kiali, Tempo and OpenTelemetry

Objectives

- Explain distributed tracing concepts including traces, spans, and trace context propagation.
- Describe the components of distributed tracing in OpenShift Service Mesh, including OpenTelemetry, Tempo, and Kiali.
- Identify automatic and manual instrumentation options for distributed tracing in OpenShift Service Mesh.
- Compare the three visualization interfaces for viewing and analyzing distributed traces: traffic graph, Observe menu, and Service Mesh workload tab.

Distributed Tracing

Distributed Tracing is the process of tracking the performance of individual services by tracing the path of service calls in an application. Each user action in an application produces a request. This request might require many services to interact to produce a response. The path of this request is a distributed transaction.

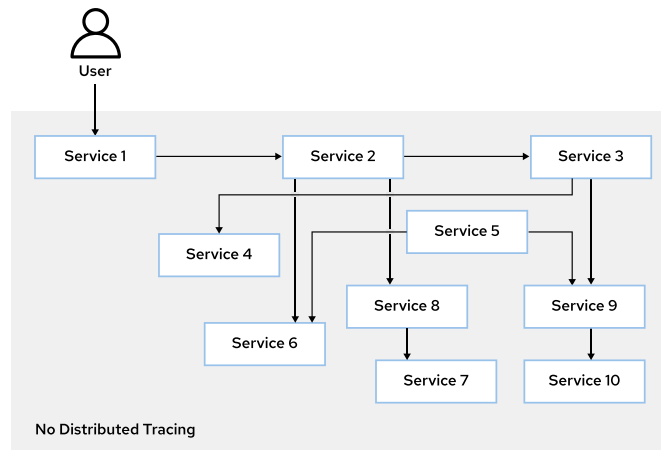


Figure 1.30: Unclear traffic flow in a system without distributed tracing

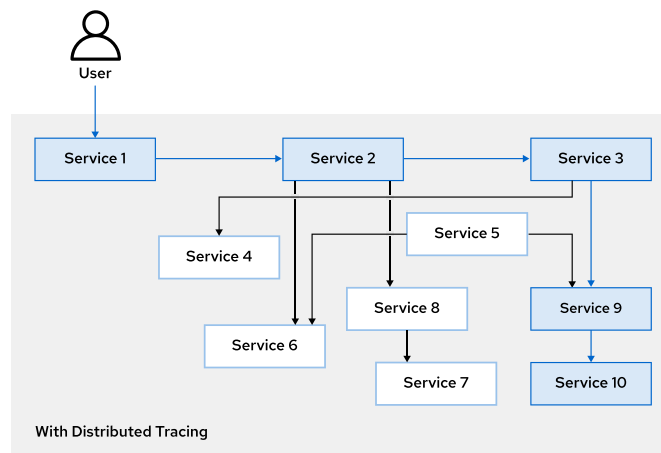


Figure 1.31: Traffic flow in a system with distributed tracing

Troubleshooting performance issues is critical for maintaining and developing distributed systems. Because a single client call can interact with multiple services, analyzing an individual service's debugging logs might not help troubleshoot performance issues.

Distributed tracing enables developers to visualize call flows in distributed applications. Understanding the sequence of calls, how many occur serially versus in parallel, and the sources of latency is useful when maintaining a distributed system.

For example, if a request takes too long, causing performance issues, then you can identify the service or services causing the slowdown and examine the network latency between service calls.

Distributed tracing is useful for monitoring, network profiling, and troubleshooting the interaction between services in modern, cloud-native, distributed applications.

Traces and Spans in Distributed Tracing

Distributed tracing uses two key terms:

Span

A *Span* represents a logical unit of work, which has a unique name, a start time, and the duration of execution. In a service mesh, service call flows are modeled as nested spans executed in a particular order.

Trace

A *Trace* is an execution path of services in the service mesh. A trace is comprised of one or more spans.

Consider an application with the following services:

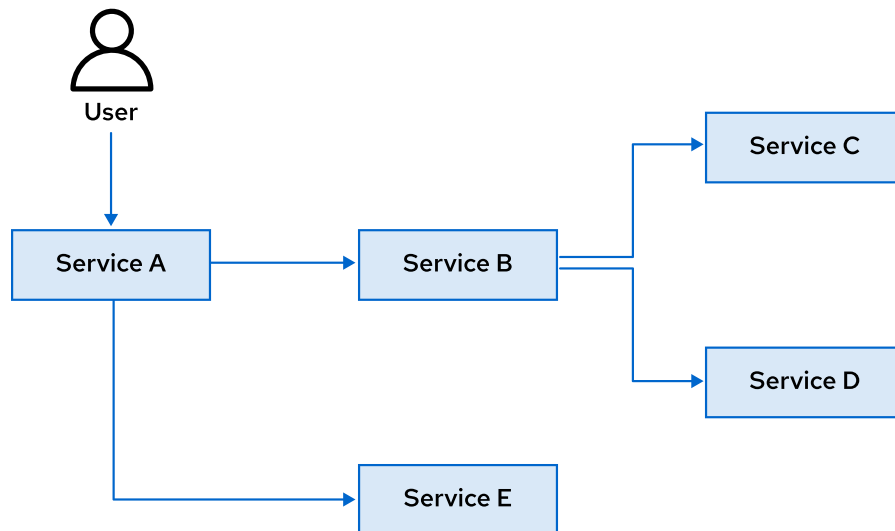


Figure 1.32: Request call path in an application composed of many components

In this example:

- Service A is the request entry point for the application.
- Because Service A is the entry point for the application, the system calls it the *parent span*. Service A makes two service calls: one to Service B and one to Service E. Thus, Service B and Service E are *child spans* of Service A.
- Service B in turn calls Service C and Service D before returning the response to Service A. Service B is a parent span. Service C and Service D are child spans.

The following is a line graph representation of the same single trace and its constituent spans:

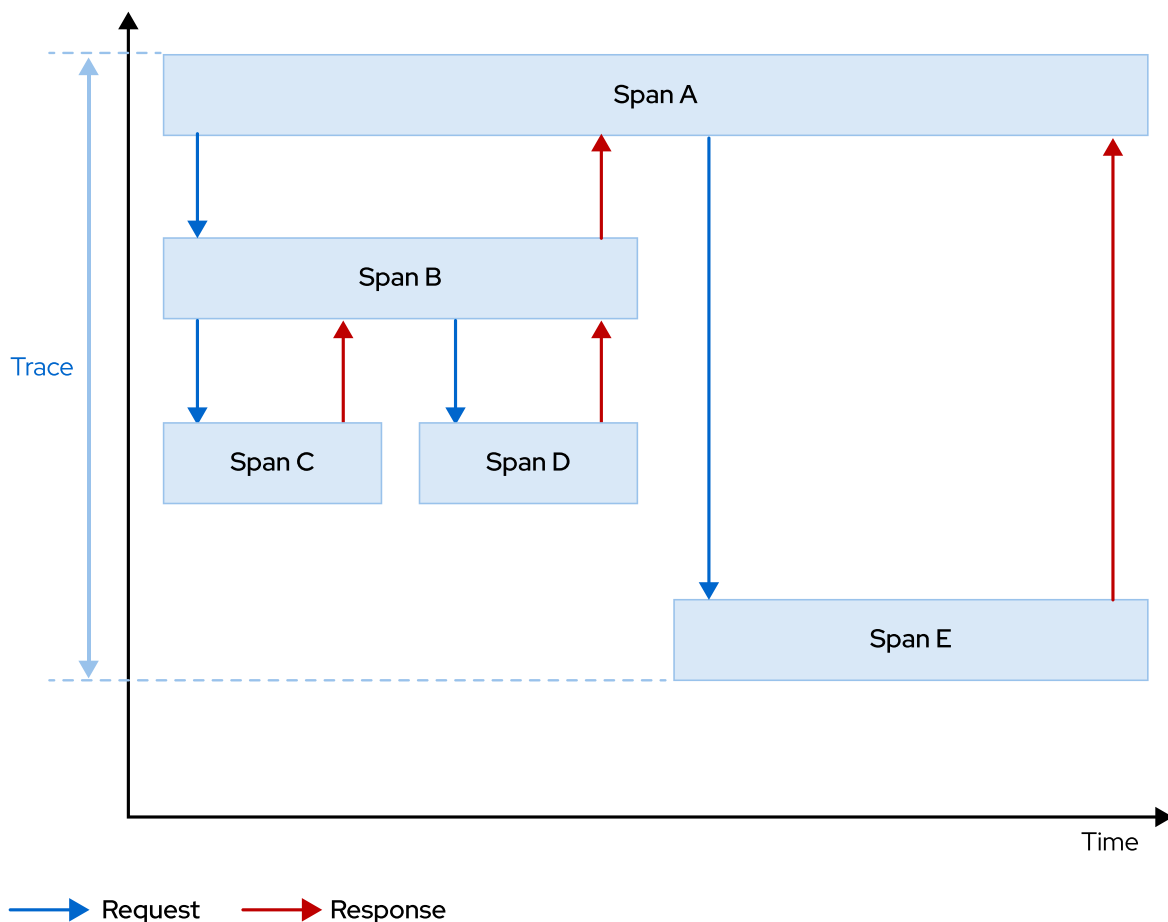


Figure 1.33: Traces and spans

Distributed Traces in Red Hat OpenShift

Red Hat OpenShift Service Mesh integrates distributed tracing capabilities through the OpenShift observability platform.

The distributed tracing solution in OpenShift Service Mesh combines three key components: OpenTelemetry, Tempo, and Kiali.

OpenTelemetry

Provides the instrumentation framework that collects trace data from services in the mesh. The Envoy sidecar proxies automatically generate and propagate trace context headers as requests flow through the mesh, creating spans for each service interaction.

Tempo

Serves as the distributed tracing backend that stores and queries trace data. As a cloud-native tracing system, Tempo efficiently handles high volumes of trace data and integrates seamlessly with the OpenShift observability stack. Tempo receives trace data from the OpenTelemetry collectors deployed in the mesh and makes this data available for visualization and analysis.

Kiali

Provides the primary visualization interface for distributed tracing in OpenShift Service Mesh. Through the OpenShift Service Mesh Console plug-in (OSSMC), Kiali integrates tracing data directly into the traffic graph and service views. Additionally, the observability

operator provides a tracing plug-in that offers an alternative interface for viewing and analyzing trace data. These visualization interfaces enable you to correlate topology, metrics, and traces in a single interface, providing a comprehensive view of application behavior and performance.

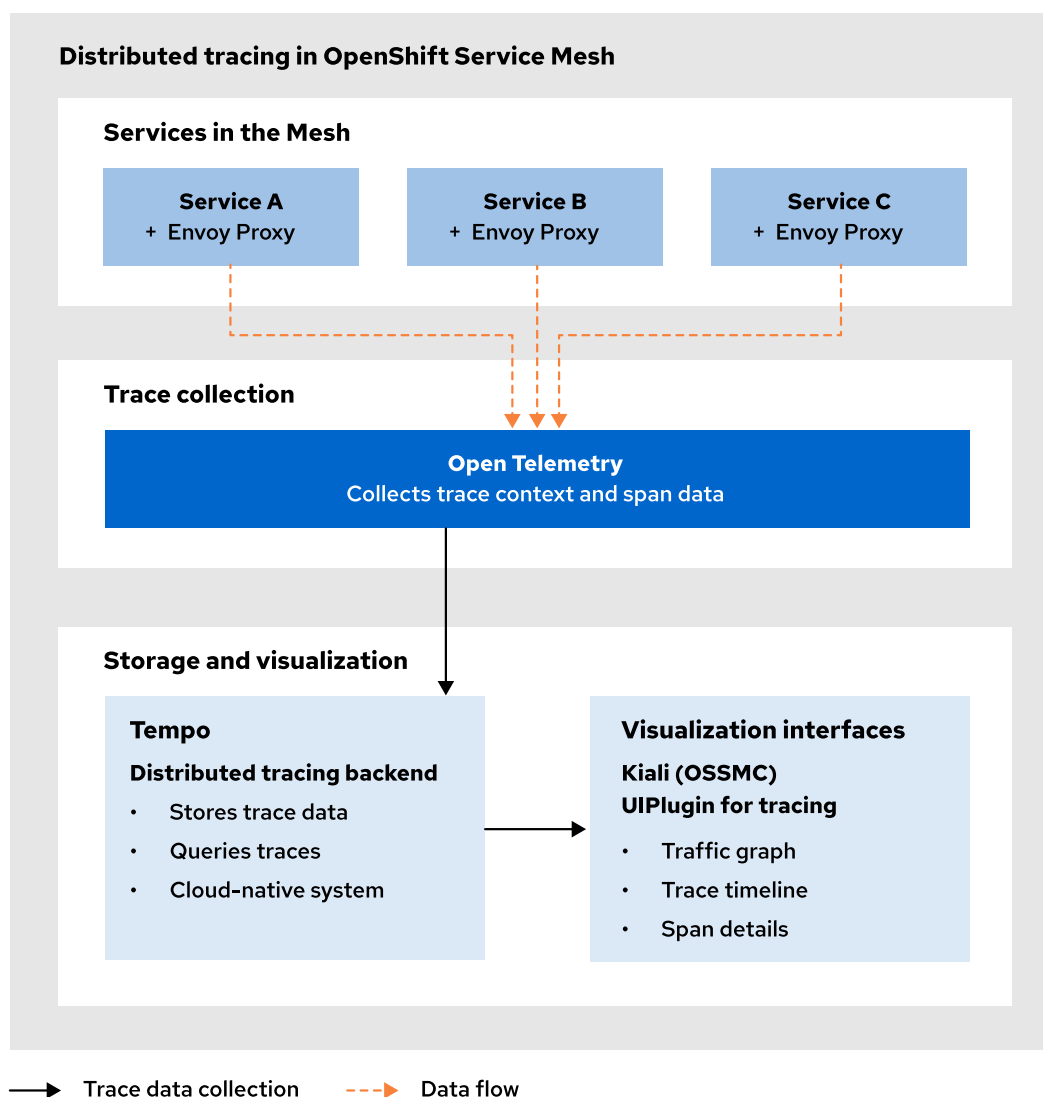


Figure 1.34: Components for distributed tracing on OpenShift and OpenShift Service Mesh

To learn more about installing and configuring the distributed tracing on OpenShift and OpenShift Service Mesh, refer to the references section.

Instrumentation Options in OpenShift Service Mesh

OpenShift Service Mesh provides automatic distributed tracing through the Envoy sidecar proxies without requiring any application code changes. The Envoy proxies automatically generate and propagate trace context headers as requests flow through the mesh, creating spans for each service-to-service interaction. This automatic instrumentation provides immediate visibility into service topology, network latency, and HTTP-level request flows across your distributed applications.

For the automatic distributed tracing, you need:

- The Tempo operator installed and configured.

- The distributed tracing UI plug-in from the Cluster Observability operator.
- The `istio-injection=enabled` label on each namespace member of the mesh.
- The PodMonitor monitoring resource to scrape metrics from the Envoy proxies on each namespace member of the mesh.

However, automatic tracing from Envoy proxies captures only network-level spans between services. To gain deeper visibility into what happens inside each service, such as database queries, internal business logic, or custom operations, you can add application-level instrumentation by using the OpenTelemetry API. Application instrumentation complements the automatic tracing provided by Envoy by creating additional spans for internal operations, adding custom attributes with business context such as order IDs or user information, and capturing application-specific errors and performance metrics. For example, a checkout for an e-commerce application involves calling from the frontend service to the checkout service. With basic instrumentation, a distributed tracing system shows the trace as a service calling another, and the duration of the operation:

```
frontend → checkout-service (234ms total)
```

If the application has instrumentation, then you can see more details for the instrumented internal operations:

```
frontend → checkout-service (234ms total)
  └─ validateCart (5ms)
  └─ checkInventory (180ms)
    └─ database.query (175ms)
  └─ calculateTax (10ms)
  └─ processPayment (35ms)
    └─ stripeAPI.charge (30ms)
  └─ createOrder (4ms)
```

In the above example, you can discover that the database query is the most time consuming operation, or span, in that trace.

OpenTelemetry provides two approaches for application instrumentation: automatic instrumentation and manual instrumentation.

Automatic instrumentation

Uses language-specific agents that require minimal code changes and automatically instrument common libraries and frameworks such as HTTP clients, database drivers, message queues, and cache systems. OpenTelemetry names the automatic instrumentation as *zero-code instrumentation*. OpenTelemetry API implementations include the following common languages:

- The `quarkus-opentelemetry` extension: OpenTelemetry implementation for the Java Quarkus framework
- The `@opentelemetry` Node.js metapackage: OpenTelemetry implementation for Node.js
- The `opentelemetry-distro` package: OpenTelemetry implementation for Python
- OpenTelemetry-Go: OpenTelemetry implementation for Go

For example, in a Node.js application, you can activate the automatic instrumentation by installing the `@opentelemetry/api` and `@opentelemetry/auto-instrumentations-node` packages, and setting environment variables:

```
[user@host application-src]$ npm install --save @opentelemetry/api
[user@host application-src]$ npm install --save @opentelemetry/auto-instrumentations-node
[user@host application-src]$ env OTEL_TRACES_EXPORTER=otlp OTEL_EXPORTER_OTLP_TRACES_ENDPOINT=opentelemetry-endpoint \
node --require @opentelemetry/auto-instrumentations-node/register app.js
```

Manual instrumentation

Gives you complete control by using the OpenTelemetry API directly in your code to create custom spans, add specific attributes, and instrument business-critical operations. The different instrumentation libraries are convenience packages that simplify manual instrumentation by providing pre-configured helpers, decorators, or middleware for specific development frameworks. These libraries require you to explicitly import them, configure them, and add instrumentation code to your application.

For example, in a Node.js application with a tracer provider configured, you can start traces and nested spans in your business logic code:

```
const { trace } = require('@opentelemetry/api'); ❶
const tracer = trace.getTracer('dice-service'); ❷

function rollOnce(min, max) {
  return Math.floor(Math.random() * (max - min + 1) + min);
}

function rollTheDice(rolls, min, max) {
  // Create a span. A span must be closed.
  return tracer.startActiveSpan('rollTheDice', (span) => { ❸
    const result = [];
    for (let i = 0; i < rolls; i++) {
      result.push(rollOnce(min, max));
    }
    // Be sure to end the span!
    span.end(); ❹
    return result;
  });
}
```

- ❶ Import the OpenTelemetry API to access tracing functionality.
- ❷ Obtain a tracer instance with a descriptive name for your service or component.
- ❸ Create a new span with a descriptive name. The span automatically becomes active in the current context.
- ❹ Always close the span when the operation completes to ensure that trace data is properly recorded and exported.

To learn more about automatic and manual instrumentation, refer to the OpenTelemetry documentation of your development language or framework in the references section.

The automatic instrumentation and manual instrumentation approaches work seamlessly with the automatic tracing provided by OpenShift Service Mesh, creating complete end-to-end traces that show both network-level service interactions and detailed application-level operations. The choice between automatic and manual instrumentation depends on your observability requirements. Automatic instrumentation provides quick setup with broad coverage of common libraries, and manual instrumentation offers precise control for instrumenting business-specific logic and adding custom context to your traces.

Trace Context Propagation

In manual instrumentation, the trace context must be propagated through the entire request path. Trace context propagation passes metadata, such as trace IDs, span IDs, and parent relationships, via HTTP headers between services, enabling the distributed tracing system to correlate all spans into a single unified trace. Without proper context propagation, each service would create independent, disconnected traces, making it impossible to understand the complete request flow through your distributed application.

The Envoy sidecar proxies in OpenShift Service Mesh automatically handle trace context propagation at the network boundaries, extracting context from incoming requests and injecting it into outgoing requests. However, when your application code makes HTTP calls to other services, the application must forward the trace context headers to maintain trace continuity. If your application uses an instrumentation library with automation capabilities, then that library propagates the headers needed to correlate traces and spans. But you can also propagate the headers manually to get basic application instrumentation.

In OpenShift Service Mesh, the tracing provider determines which headers to use. For example, if the tracing provider is Zipkin, as in previous versions of OpenShift Service Mesh, then you must propagate B3 headers, such as `x-b3-traceid`, `x-b3-spanid`, or `x-b3-parentspanid`. Red Hat recommends to use OpenTelemetry for distributed tracing. OpenTelemetry uses the W3C trace context format, which needs to propagate the following headers:

- `traceparent`
- `tracestate`

To inspect which tracing provider your OpenShift Service Mesh installation uses, read the `istio` configuration map within the namespace where Istio is running. The `extensionProviders` field contains the tracing provider.



Figure 1.35: OpenTelemetry tracing provider in the Istio configuration map

Visualizing Traces: Traffic Graph Integration

The OpenShift web console provides multiple interfaces for viewing and analyzing distributed traces.

The OpenShift Service Mesh traffic graph in the OpenShift web console provides a topology view of service mesh traffic. From this graph, you can drill down to inspect distributed traces for specific services.

First, click any service node in the graph to view its details.

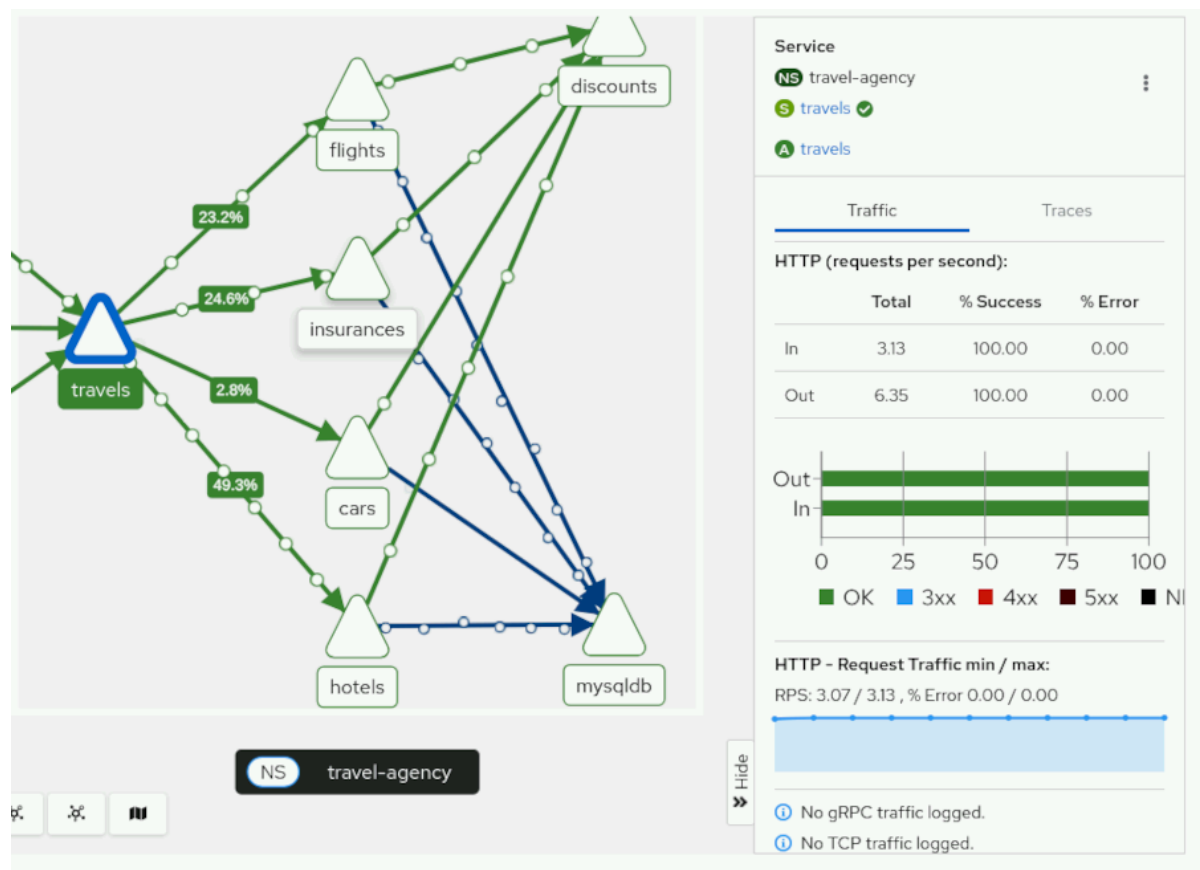


Figure 1.36: Traffic graph after clicking a service node

Select the Traces tab in the service detail panel to view a list of traces that involve the selected service.

When you click one of the traces, the traffic graph shows with a blue line the path that the trace did. Also, a subpanel opens within the service detail panel. This subpanel contains the trace information.

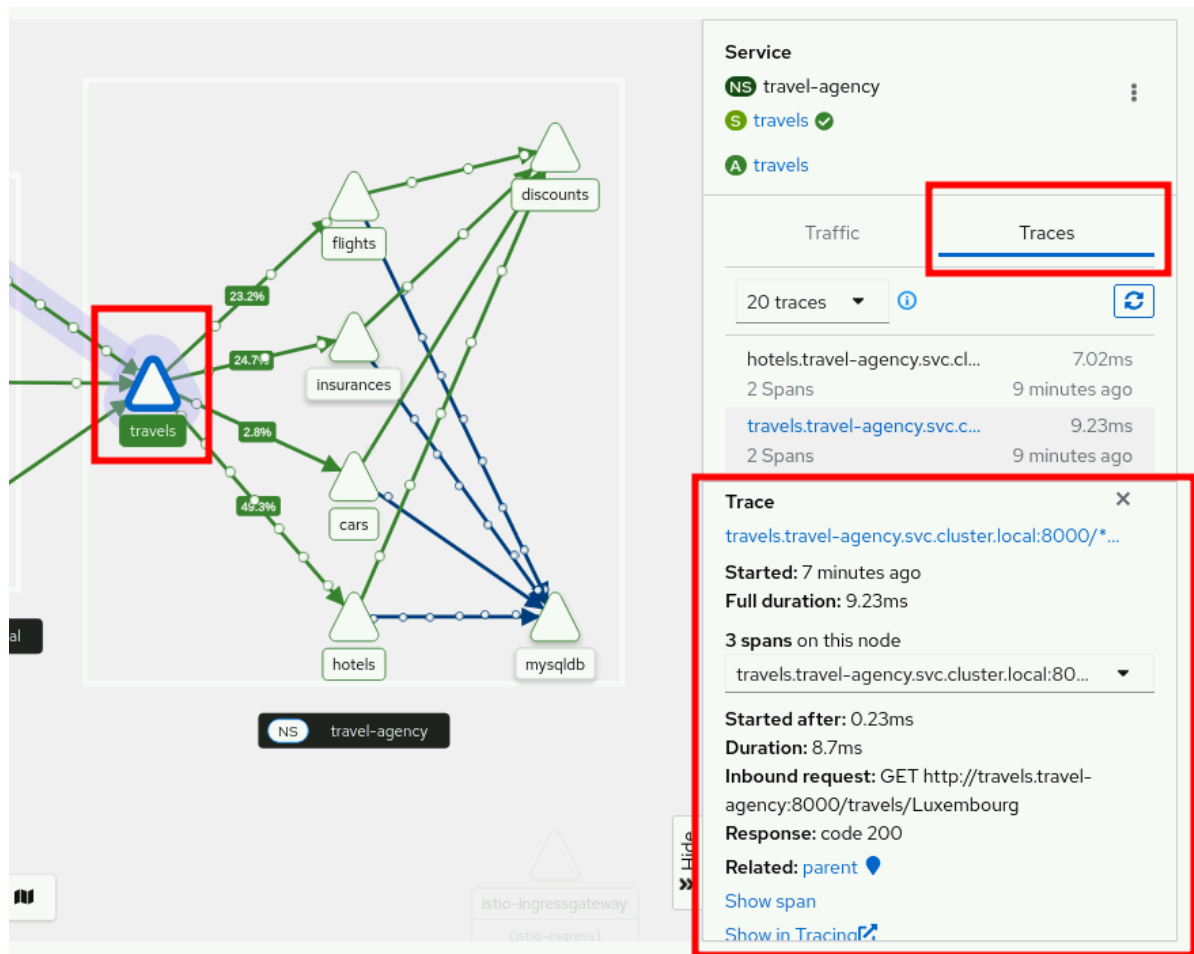


Figure 1.37: The service detail panel containing the trace subpanel

This integration enables you to start from the topology view and drill down to traces for specific services, helping you investigate performance issues by first identifying problematic services in the graph, and then examining their specific trace patterns.

Visualizing Traces: The OpenShift Console Observe Menu

The OpenShift web console provides a dedicated interface for traces through the Observe → Traces menu. The UI plug-in component from the Red Hat Observability operator provides this interface.

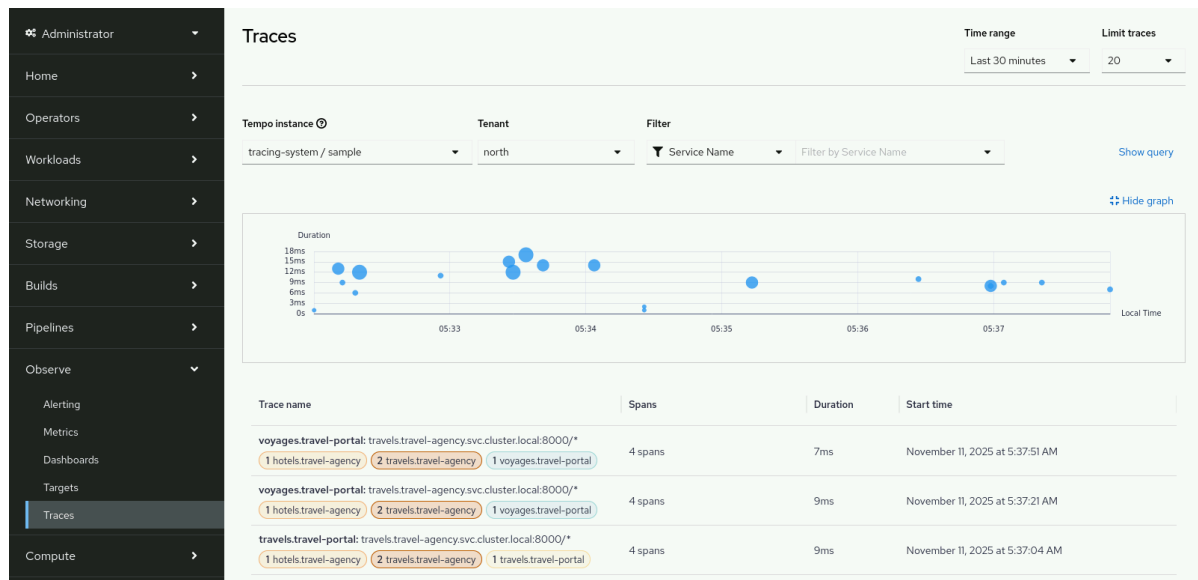


Figure 1.38: Observe traces view in OpenShift console

From this interface, you can:

- Search and filter traces: Query traces by service name, operation, tags, duration, and time range to find specific requests or patterns.
- View trace lists: Browse all traces collected in your cluster, sorted by different criteria such as timestamp or duration.
- Analyze trace distributions: Examine histograms and scatter plots showing trace duration distributions to identify outliers and performance trends.
- Deep-dive into specific traces: Select individual traces to view their complete span hierarchy and timing details.



Figure 1.39: Span hierarchy and timing details view

The trace detail view shows a timeline visualization showing:

- Each span as a horizontal bar
- The duration of each span, represented by the length of the bar

- Parent-child relationships between spans, indicated by indentation or tree structure
- The sequence and timing of service calls

This visualization helps you understand:

- Which service contributes most to the total response time
- Whether calls happen in parallel or sequentially
- Sources of latency in the request path
- Service dependencies and interaction patterns

The Observe menu provides a broader view of distributed tracing data across your entire OpenShift cluster, complementing the service-specific trace views available in the **Service Mesh** tab from each workload.

Visualizing Traces: Service Mesh Workload and Service Traces Tab

The OpenShift Service Mesh Console plug-in adds a **Service Mesh** tab to many resource types, such as services, deployments, stateful sets, or daemon sets in the OpenShift web console.

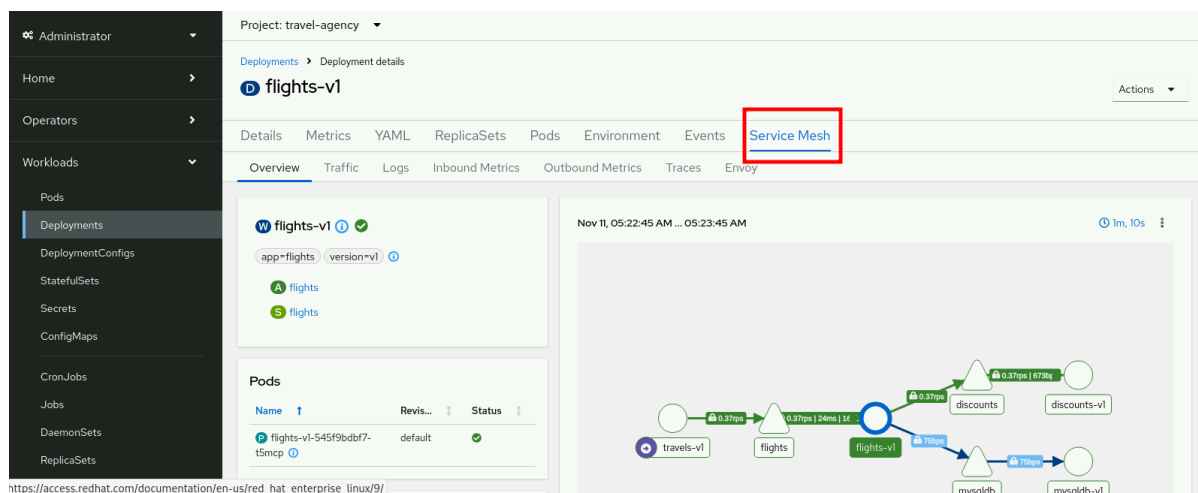


Figure 1.40: Service mesh tab in an OpenShift resource

Within this tab, there is another sub tab, **Traces**.

From the **Traces** tab for a specific workload, you can:

- View all traces for the workload: To see traces where this workload is either the entry point or a participant in the request path.
- Filter traces by direction: To focus on inbound traces, which are requests received by the workload, or outbound traces, which are requests initiated by the workload.
- Examine trace details: Click any trace to view its complete span hierarchy, showing how the request flows through the service mesh.
- Analyze workload performance: Identify the workload's contribution to overall request latency by examining span durations.

- Investigate errors: Filter for traces containing errors to troubleshoot failures specific to this workload.
- Correlate with metrics: View traces alongside the workload's metrics in the Service Mesh tab to understand the relationship between aggregate performance and individual requests.

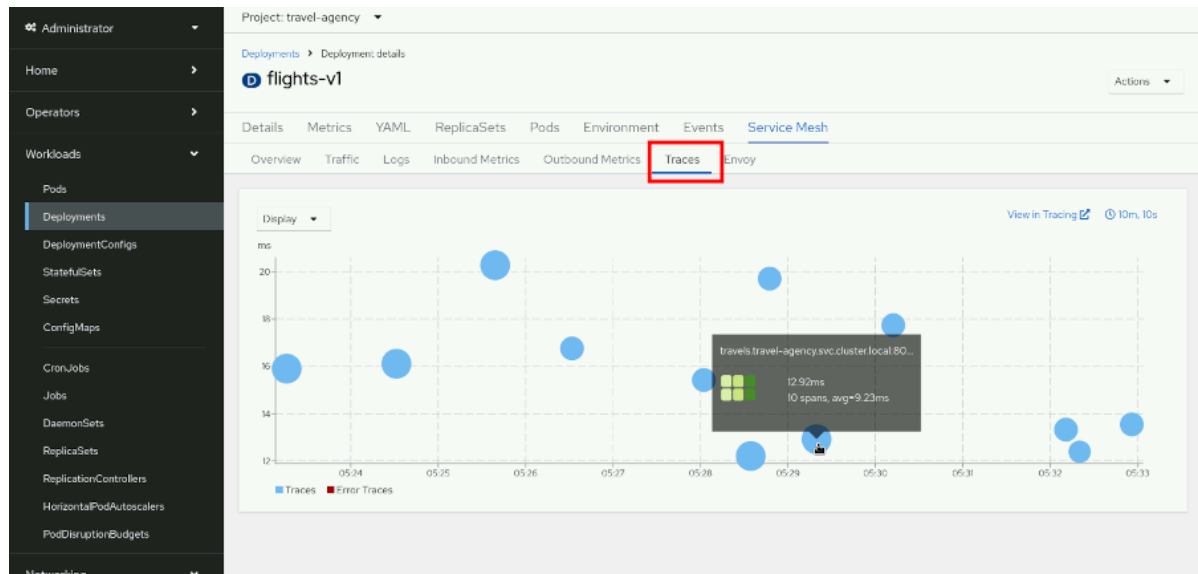


Figure 1.41: Traces tab view in a deployment resource

This workload-specific trace view is particularly useful when:

- Troubleshooting performance or reliability issues for a specific service version.
- Understanding how a particular workload interacts with other services in the mesh.
- Validating that configuration changes, such as retry policies or circuit breakers, affect trace patterns as expected.
- Investigating why a specific workload exhibits different behavior than other versions.

The Traces tab in the Service Mesh workload view provides a focused, workload-centric perspective on distributed tracing, complementing the topology-based view in the traffic graph and the cluster-wide view in the Observe menu.

The OpenShift web console provides three different interfaces for viewing distributed traces, each optimized for specific troubleshooting scenarios. The following table summarizes when to use each interface:

Visualization	Best For	Primary Use Cases
Traffic Graph	Topology-based investigation	- Identifying problematic services in the mesh topology - Understanding service-to-service communication patterns - Correlating traffic flow with trace data - Investigating issues across multiple services
Observe Menu	Cluster-wide trace analysis	- Searching traces across all services and namespaces - Analyzing trace duration distributions and performance trends - Finding specific requests by service name, operation, or tags - Examining historical trace data
Service Mesh Workload Tab	Workload-specific troubleshooting	- Debugging a specific service version or deployment - Validating configuration changes for a particular workload - Investigating performance differences between workload versions - Analyzing inbound and outbound traffic for a single workload

Each interface complements the others to provide a comprehensive view of distributed tracing in your service mesh.

Correlating Metrics and Traces

The OpenShift web console integrates metrics and traces to provide a comprehensive view of application behavior:

- The traffic graph shows overall traffic patterns and service relationships.
- Metrics show aggregate performance data.
- Traces show individual request flows and detailed timing information.

This correlation helps you with the following activities:

- Identify performance anomalies in metrics

- Investigate specific requests that contribute to those anomalies.
- Understand how traffic distribution across service versions affects performance.
- Troubleshoot issues by examining both aggregate patterns and individual requests.

Using Traces for Troubleshooting

Distributed tracing helps answer critical troubleshooting questions:

- Which service is causing performance degradation?
- Why are some requests slower than others?
- What is the sequence of service calls for a failing request?
- Where are errors occurring in the request path?
- How does network latency between services affect response time?

By examining traces, you can:

1. Identify the service or services contributing to high latency
2. Examine the timing of parallel versus sequential calls
3. Locate errors and exceptions in specific spans
4. Correlate application logs with trace data
5. Understand the impact of service mesh configuration on request flow

These insights help you understand application behavior, optimize performance, and troubleshoot issues in a distributed architecture.

REFERENCES

For more general information about the distributed tracing platform on OpenShift, refer to the *Distributed Tracing* guide in the Red Hat OpenShift documentation at

https://docs.redhat.com/en/documentation/openshift_container_platform/4.18/html-single/distributed_tracing/index
(https://docs.redhat.com/en/documentation/openshift_container_platform/4.18/html-single/distributed_tracing/index)

For more information about installing and configuring the distributed tracing on OpenShift, refer to the *Installing the Distributed Tracing Platform* chapter in the *Distributed Tracing* guide in the Red Hat OpenShift documentation at https://docs.redhat.com/en/documentation/openshift_container_platform/4.18/html-single/distributed_tracing/index#distr-tracing-tempo-installing (https://docs.redhat.com/en/documentation/openshift_container_platform/4.18/html-single/distributed_tracing/index#distr-tracing-tempo-installing)

For more information about configuring the distributed tracing on OpenShift Service Mesh, refer to the *Distributed Tracing and Service Mesh* chapter in the *Observability* guide in the Red Hat OpenShift Service Mesh documentation at

https://docs.redhat.com/en/documentation/red_hat_openshift_service_mesh/3.1/html-single/observability/index#ossm-distr-tracing

([https://docs.redhat.com/en/documentation/red_hat_openshift_service_mes
h/3.1/html-single/observability/index#ossm-distr-tracing](https://docs.redhat.com/en/documentation/red_hat_openshift_service_mesh/3.1/html-single/observability/index#ossm-distr-tracing))

The Path to Distributed Tracing: an OpenShift Observability Adventure

([https://www.redhat.com/en/blog/the-path-to-distributed-tracing-an-
openshift-observability-adventure](https://www.redhat.com/en/blog/the-path-to-distributed-tracing-an-openshift-observability-adventure))

The path to distributed tracing: an OpenShift Observability adventure part II: A twist in the myth ([https://www.redhat.com/en/blog/path-distributed-tracing-
openshift-observability-adventure-part-ii-twist-myth](https://www.redhat.com/en/blog/path-distributed-tracing-openshift-observability-adventure-part-ii-twist-myth))

The Path to Distributed Tracing: a Red Hat OpenShift Observability Adventure part 3 Building the bridge ([https://www.redhat.com/en/blog/the-path-to-
distributed-tracing-part-3-building-the-bridge](https://www.redhat.com/en/blog/the-path-to-distributed-tracing-part-3-building-the-bridge))

Using OpenTelemetry Tracing Quarkus Guide

(<https://quarkus.io/guides/opentelemetry-tracing>)

Getting Started Node.js OpenTelemetry Guide

(<https://opentelemetry.io/docs/languages/js/instrumentation/>)

Getting Started Python OpenTelemetry Guide

(<https://opentelemetry.io/docs/languages/python/getting-started/>)

Getting Started Go OpenTelemetry Guide

(<https://opentelemetry.io/docs/languages/go/getting-started/>)

Revision: ad00071-3.1-ed3-20260313-9c0e3a4



개인 정보 취급 방침 (https://www.redhat.com/ko/about/privacy-policy?extIdCarryOver=true&sc_cid=701f20000001D8QoAAK)

이용 약관 (<https://www.redhat.com/ko/about/terms-use>)

릴리스 노트

(https://docs.redhat.com/en/documentation/red_hat_learning_subscription/1-latest/html-single/red_hat_learning_subscription_release_notes/index)

Red Hat 교육 정책

(<https://www.redhat.com/ko/about/red-hat-training-policies>)

모든 정책 및 지침

(<https://www.redhat.com/ko/about/all-policies-guidelines>)

쿠키 기본설정